

Metabolite Set Enrichment Analysis

ORA and MSEA for metabolomics

Alex Sanchez

2026-03-20

Contents

Introduction	2
Data and metabolite sets	2
The datasets	3
Loading datasets	3
Metabolite Sets	5
Load Metabolite Sets	6
Building signatures and backgrounds	9
Cachexia: HMDB mapping and signature	9
ST000291: PubChem and HMDB mappings and signatures	11
ST000291 with PubChem identifiers	11
ST000291 with HMDB identifiers	12
Enrichmet Example: KEGG mappings and signature	13
Enrichment analysis of the Cachexia data	13
Over Representation Analysis (ORA)	13
ORA with <code>localEnrichment</code> and <code>KEGGsets_HMDB</code>	13
ORA with <code>enrichmet</code> and <code>KEGGsets_HMDB</code>	15
Metabolite Set Enrichment Analysis	16
MSEA with <code>localEnrichment</code> and <code>KEGGsets_HMDB</code>	16
MSEA with <code>enrichmet</code> and <code>KEGGsets_HMDB</code>	17
MSEA with <code>enrichmet</code> and <code>KEGGsets_KEGG</code>	17
Comparison of results	19
Enrichment analysis of the enrichmet example data (KRASG12D)	19
ORA with <code>localEnrichment</code> and <code>KEGGset_KEGG</code>	19
ORA with <code>enrichmet</code> and <code>EM_PathwayVsMetabolites</code>	21
MSEA / QEA with <code>localEnrichment</code> and <code>KEGGset_KEGG</code>	23
MSEA / GSEA with <code>enrichmet</code>	24
Quick comparison of the top pathways	24
Appendix I Methods and packages explained	25
Over-Representation Analysis (ORA)	25
Concept	25
Implementation in <code>localEnrichment::ora_test()</code>	26
Quantitative Enrichment Analysis (QEA / MSEA)	26
Concept	26
MSEA variant implemented in <code>localEnrichment</code>	27

Resources and references	27
Resources	27
References	27

Introduction

Enrichment analysis aims to facilitate the biological interpretation of omics results by relating lists of features (genes, proteins, metabolites) to prior biological knowledge stored in databases (pathways, ontologies, chemical classes, etc.).

In genomics, ORA and GSEA workflows are relatively well standardized, and code-based (`clusterProfiler` or graphical-interface (`g-profiler` or `Reactome`) are well established tools.

In metabolomics, however, the scenario is more-fragmented and available tools take more heterogeneous approaches and rely on different identifier systems and pathway databases.

The goals of this document are:

1. **Quick-start:** show how to run basic over-representation analysis (ORA) and metabolite set enrichment analysis (MSEA/GSEA-like) with several tools.
2. **Naïve benchmarking:** compare the results of these tools on the same datasets.

We will focus on:

- `enrichmet` (pathway enrichment, GSEA, centrality, plots)
- `localEnrichment` (our own infrastructure for metabolite sets and ORA/MSEA)
- `ChemRich` (https://raw.githubusercontent.com/barupal/ChemRICH/refs/heads/master/chemrich_minimum_analysis.R)
- `MetaboAnalystR`, has become very hard to make it work so the examples will be run from its web site and downloaded.

Rather than exhaustively exploring each tool, we restrict ourselves to a reproducible workflow:

1. We have selected a few public metabolite datasets. The selection has been based on “popularity” and availability, but the workflow should be *easily adaptable to new datasets*.
2. We have prepared an **ad-hoc collection of metabolite sets**, that is, Metabolite sets that have been programatically prepared, as `data.frames` and `EnrichmentSet` (an S4 class created to store Metabolite Sets Collections), from the `Lheuristic` package, to be used for these analyses.
3. For each dataset we have prepared:
 - A list of **significant metabolites** (*signature*),
 - A **background** tailored to that signature created either ad-hoc or from the original
 - An **effect size** table with summary statistics that can be used for MSEA tests.
4. Given these components we will run enrichment analysis on them (ORA and MSEA when possible) with the different tools on the same sets.
5. As more analyses are available we will increase comparisons among results to check which analyses are related and which lead different results because they are indeed different analyses.

Data and metabolite sets

Any pathway analysis requires, at least, two inputs:

- A list of features usually associated with a phenotype (e.g. metabolites differentially quantified between two groups).

- A source of annotations for these features, that can be generically described as *Feature sets* or, as in this case *Metabolite Sets*.

The datasets

We use three datasets that have been preprocessed elsewhere (see file: PrepareData-4-EnrichmentAnalysis.Rmd) and

1. The **ia** dataset. This is a popular example used by **MetaboAnalyst**, where a comparison between affected and control patients is performed. It has the drawback that only *differential* metabolites are provided, which makes it only partially interesting for some types of analyses. We keep it for its popularity.
2. The **ST000291** dataset, downloaded from Metabolomics Workbench, and used in a **fobitools** use case. This is obtained by a nutritional intervention study where a comparison among consumption of Apple, Cranberries and Baseline has been performed. In this case we have the full dataset and the results of comparing the groups using **limma** which leads to around 20 differential metabolites. Here, we focus only on the *Cranberry vs Baseline* comparison, although lists from other comparisons are also available and may be used when a compare-comparisons approach is considered. This study seems to have been made using a cross-over design that the authors have later omitted when providing the samples. Because it is only for instructional purposes, this flaw will be ignored.
3. The *KrasG12D mutation dataset*, a list of cancer related metabolites used by the **Enrichmet** package in one of its tutorial.

Loading datasets

The data for this use case has been prepared elsewhere and is available in the **data** directory.

For each dataset we have the list of metabolites, the analysis results in the form of a topTable and a mapping performed with MetaboAnalyst between the metabolite ids and several databases as shown below.

```
# Cachexia: topTable + ID mapping
cachexia_map_path <- "data/Cachexia_map.csv"
cachexia_tt_path  <- "data/Cachexia_topTable.Rda"

Cachexia_map <- read.csv(cachexia_map_path, stringsAsFactors = FALSE)
load(cachexia_tt_path) # loads 'topCachexia'

cat("Cachexia_map:\n")
```

Cachexia dataset

```
## Cachexia_map:
```

```
print_tbl(head(Cachexia_map))
```

```
## # A tibble: 6 x 9
##   Query      Match      HMDB PubChem ChEBI KEGG  METLIN SMILES Comment
##   <chr>      <chr>      <chr>   <int> <chr> <chr>   <int> <chr>   <int>
## 1 2-Hydroxyisobutyrate 2-Hydrox~ HMDB~  4277439 19641 <NA>      NA CC(C)~      1
## 2 Creatinine      Creatini~ HMDB~    588 16737 C007~      8 CN1CC~      1
## 3 Quinolinat~    Quinol~ HMDB~   1066 16675 C037~   330 OC(=O~      1
## 4 Valine      L-Valine HMDB~   6287 16414 C001~   5842 CC(C)~      1
## 5 Dimethylamine Dimethyl~ HMDB~    674 17170 C005~   3758 CNC       1
## 6 Tryptophan    L-Trypt~ HMDB~   6305 16828 C000~   5879 N[C@@~      1
```

```
cat("\ntopCachexia:\n")
```

```
##
```

```
## topCachexia:
```

```
print_tbl(head(topCachexia))
```

```
## # A tibble: 6 x 7
```

```
##   logFC AveExpr      t P.Value adj.P.Val      B ordByLimma
##   <dbl>  <dbl> <dbl>   <dbl>   <dbl> <dbl>   <int>
## 1  44.4    66.4  4.06 0.000119 0.00426 -3.92      1
## 2  25.4    35.7  3.97 0.000164 0.00426 -3.95      2
## 3  20.9    26.3  3.90 0.000203 0.00426 -3.97      3
## 4  17.7    24.4  3.72 0.000378 0.00431 -4.02      4
## 5  245.    358.  3.69 0.000417 0.00431 -4.03      5
## 6  151.    211.  3.67 0.000455 0.00431 -4.04      6
```

```
# ST000291: Cranberry vs Baseline topTable + ID mapping
```

```
st291_map_path <- "data/MW_ST000291_map.csv"
```

```
st291_tt_path <- "data/MW_ST000291-limma_res.Rda"
```

```
MS_ST000291_map <- read.csv(st291_map_path, stringsAsFactors = FALSE)
```

```
load(st291_tt_path) # loads 'topCranVSBase'
```

```
topCranVSBase<- limma_resCB; rm(limma_resCB)
```

```
topCranVSBase_HMDB<- limma_HMDB_CB; rm(limma_HMDB_CB)
```

```
cat("\nMS_ST000291_map:\n")
```

Metabolomics Workbench ST000291 dataset

```
##
```

```
## MS_ST000291_map:
```

```
print_tbl(head(MS_ST000291_map))
```

```
## # A tibble: 6 x 9
```

```
##   Query Match          HMDB PubChem ChEBI KEGG METLIN SMILES Comment
##   <int> <chr>          <chr>   <int> <int> <chr>   <int> <chr>   <int>
## 1  192798 Digitalose    <NA>   192798    NA <NA>     NA <NA>     1
## 2   79034 12-Hydroxydodecanoic~ HMDB~   79034 39567 C083~   6465 OCCCC~   1
## 3   10413 4-Hydroxybutyric acid HMDB~   10413 30830 C009~   5678 OCCCC~   1
## 4   439230 Mevalonic acid HMDB~  439230 17710 C004~   127 C[C@@~   1
## 5  20975673 <NA>          <NA>     NA    NA <NA>     NA <NA>     0
## 6   5275508 <NA>          <NA>     NA    NA <NA>     NA <NA>     0
```

```
cat("\ntopCranVSBase:\n")
```

```
##
```

```
## topCranVSBase:
```

```
print_tbl(head(topCranVSBase))
```

```
## # A tibble: 6 x 7
```

```
##   PubChemCID log2FC AveExpr      t      pvalue adj_pvalue      B
##   <chr>      <dbl>  <dbl> <dbl>   <dbl>   <dbl> <dbl>
## 1 54678503    2.92 3.57e-16 6.45 0.0000000623 0.0000847 8.11
```

```
## 2 71485      3.27 -2.40e-16  5.85 0.000000484  0.000329  6.19
## 3 94214      2.18 -8.93e-17  5.69 0.000000843  0.000382  5.68
## 4 5378303    2.41  1.10e-16  5.54 0.00000140  0.000403  5.20
## 5 3037       3.18  8.39e-17  5.46 0.00000184  0.000403  4.95
## 6 10178      3.33  3.81e-17  5.45 0.00000189  0.000403  4.92
```

```
cat("\ntopCranVSBase (HMDB):\n")
```

```
##
```

```
## topCranVSBase (HMDB):
```

```
print_tbl(head(topCranVSBase_HMDB))
```

```
## # A tibble: 6 x 7
```

##	feature	log2FC	AveExpr	t	pvalue	adj_pvalue	B
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
## 1	HMDB0251200	3.18	8.39e-17	5.48	0.00000172	0.000876	5.01
## 2	HMDB0248806	2.26	3.85e-17	5.34	0.00000281	0.000876	4.54
## 3	HMDB0258615	2.05	2.68e-16	5.00	0.00000881	0.00154	3.47
## 4	HMDB0006115	1.56	-4.28e-16	4.96	0.00000986	0.00154	3.36
## 5	HMDB0000738	1.46	1.01e-15	4.72	0.0000221	0.00233	2.60
## 6	HMDB0000714	1.51	-8.74e-16	4.72	0.0000225	0.00233	2.58

```
enrichmet_path <- "data/enrichmet_example.Rda"
load(enrichmet_path) # loads 'topCranVSBase'
cat("\nEM_summaryStats:\n")
```

enrichmet package example dataset

```
##
```

```
## EM_summaryStats:
```

```
print_tbl(head(EM_summaryStats))
```

```
## # A tibble: 6 x 5
```

##	met_id	pval	padj	log2fc	met_id_raw
##	<chr>	<dbl>	<dbl>	<dbl>	<chr>
## 1	C02721	0.0000124	0.00315	-2.32	C02721
## 2	C03736	0.0000694	0.00739	2.63	C03736
## 3	C03139	0.0000869	0.00739	1.75	C03139
## 4	C00074	0.000120	0.00763	-3.22	C00074
## 5	C02862	0.000198	0.00819	-1.70	C02862
## 6	C00386	0.000225	0.00819	1.68	C00386

```
# cat("\nEM_mapping_df:\n")
# print_tbl(head(EM_mapping_df))
# cat("\nEM_kegg_lookup:\n")
# print_tbl(head(EM_kegg_lookup))
# cat("\nEM_PathwayVsMetabolites:\n")
# print_tbl(head(EM_PathwayVsMetabolites))
# cat("\nKEGGset_KEGG:\n")
# show(KEGGset_KEGG)
```

Metabolite Sets

We also use a **unified collection of metabolite sets**, previously constructed and saved as:

- KEGGset_HMDB, KEGGset_HMDB_df
- KEGGset_PubChem, KEGGset_PubChem_df
- ChemicalClassSet, ChemicalClassSet_df
- SMPDBset, SMPDBset_df

All sets collections, but KEGGset_PubChem use **HMDB IDs** as feature identifiers. KEGGset_PubChem uses **PubChemIDs**.

Load Metabolite Sets

Metabolite sets have been prepared elsewhere and saved as **EnrichmentSet** objects of the class defined in the **localEnrichment** package.

```
# Metabolite sets (EnrichmentSet collection)
# mets_collection_path <- "databases/MetaboliteSets_Collection.Rda"
# load(mets_collection_path)

load("../Metabolites_Sets/results/MetaboliteSets_Collection.Rda")

### Parche. Pendent de netejar-ho al script que ho crea
KEGGset_HMDB_df <- df_HMDB
KEGGset_PubChem_df <- df_PubChem

###

cat("\nEnrichmentSet objects loaded:\n")
```

```
##
```

```
## EnrichmentSet objects loaded:
```

```
summary(KEGGset_HMDB);      cat("\n")
```

```
## EnrichmentSet summary:
```

```
## Mapping name: KEGG_pathways_hsa_HMDB
```

```
## Source: KEGG
```

```
## Feature IDs: HMDB
```

```
## Number of sets: 283
```

```
## Mean set size: 22.5689 features
```

```
## Median set size: 10 features
```

```
summary(KEGGset_PubChem);   cat("\n")
```

```
## EnrichmentSet summary:
```

```
## Mapping name: KEGG_pathways_hsa_PubChem
```

```
## Source: KEGG
```

```
## Feature IDs: PubChem
```

```
## Number of sets: 267
```

```
## Mean set size: 18.15356 features
```

```
## Median set size: 8 features
```

```
summary(ChemicalClassSet);  cat("\n")
```

```
## EnrichmentSet summary:
```

```
## Mapping name: ChemicalClasses
```

```
## Source: HMDB
```

```
## Feature IDs: HMDB
```

```
## Number of sets: 31
```

```
## Mean set size: 13.54839 features
```

```

## Median set size: 4 features
summary(SMPDBset);          cat("\n")

## EnrichmentSet summary:
## Mapping name: SMPDB_pathways
## Source: SMPDB
## Feature IDs: HMDB
## Number of sets: 48654
## Mean set size: 17.30571 features
## Median set size: 18 features

cat("\nKEGGset_HMDB_df:\n")

##
## KEGGset_HMDB_df:
print_tbl(head(KEGGset_HMDB_df))

## # A tibble: 6 x 3
##   set_name                set_id  Metabolites
##   <chr>                  <chr>    <chr>
## 1 Glycolysis / Gluconeogenesis hsa00010 HMDB00000243;HMDB0001206;HMD~
## 2 Citrate cycle (TCA cycle)    hsa00020 HMDB00000243;HMDB0001206;HMD~
## 3 Pentose phosphate pathway    hsa00030 HMDB00000243;HMDB0000124;HMD~
## 4 Pentose and glucuronate interconversions hsa00040 HMDB00000243;HMDB0000208;HMD~
## 5 Fructose and mannose metabolism hsa00051 HMDB00000124;HMDB0001163;HMD~
## 6 Galactose metabolism         hsa00052 HMDB00000286;HMDB0000302;HMD~

cat("\nKEGGset_PubChem_df:\n")

##
## KEGGset_PubChem_df:
print_tbl(head(KEGGset_PubChem_df))

## # A tibble: 6 x 3
##   set_name                set_id  Metabolites
##   <chr>                  <chr>    <chr>
## 1 Glycolysis / Gluconeogenesis hsa00010 1060;107735;176;175;970;906~
## 2 Citrate cycle (TCA cycle)    hsa00020 1060;107735;51;164533;970;1~
## 3 Pentose phosphate pathway    hsa00030 1060;107735;439168;5311110;~
## 4 Pentose and glucuronate interconversions hsa00040 1060;107735;51;164533;8629;~
## 5 Fructose and mannose metabolism hsa00051 439709;668;439168;107689;79~
## 6 Galactose metabolism         hsa00052 8629;5988;439709;668;753;43~

cat("\nChemicalClassSet_df:\n")

##
## ChemicalClassSet_df:
print_tbl(head(ChemicalClassSet_df))

## # A tibble: 6 x 3
##   set_name                set_id  Metabolites
##   <chr>                  <chr>    <chr>
## 1 Acylcarnitines          Acylcarnitines HMDB00000062;HMDB0000201;HMDB0~
## 2 Amine oxide             Amine oxide    HMDB00000925
## 3 Amino Acids             Amino Acids    HMDB00000161;HMDB00000517;HMDB0~

```

```
## 4 Amino Acids Derivatives Amino Acids Derivatives HMDB0000092;HMDB0000766;HMDB0~
## 5 Amino acid - related Amino acid - related HMDB0000510;HMDB0000452;HMDB0~
## 6 Azoles Azoles HMDB0000462
```

```
cat("\nSMPDBset_df:\n")
```

```
##
```

```
## SMPDBset_df:
```

```
print_tbl(head(SMPDBset_df))
```

```
## # A tibble: 6 x 3
```

##	set_name	set_id	Metabolites
##	<chr>	<chr>	<chr>
## 1	Citrullinemia Type I	SMP0000001	HMDB0000641;HMDB0000161;~
## 2	Carbamoyl Phosphate Synthetase Deficiency	SMP0000002	HMDB0000641;HMDB0000161;~
## 3	Argininosuccinic Aciduria	SMP0000003	HMDB0000641;HMDB0000161;~
## 4	Glycine and Serine Metabolism	SMP0000004	HMDB0002134;HMDB0001377;~
## 5	Pterine Biosynthesis	SMP0000005	HMDB0006822;HMDB0002111;~
## 6	Tyrosine Metabolism	SMP0000006	HMDB0000158;HMDB0000208;~

The Metabolite sets for the `Enrichmet` package example data have been loaded when loading the example data but are shown here for consistency.

They consist of a `data.frame` and an `EnrichmentSet` object that has been prepared from this

```
cat("\nEM_PathwayVsMetabolites:\n")
```

```
##
```

```
## EM_PathwayVsMetabolites:
```

```
print_tbl(head(EM_PathwayVsMetabolites))
```

```
## # A tibble: 6 x 2
```

##	Pathway	Metabolites
##	<chr>	<chr>
## 1	Glycolysis / Gluconeogenesis	C00022,C00024,C00031,C00033,C00036,C~
## 2	Citrate cycle (TCA cycle)	C00022,C00024,C00026,C00036,C00042,C~
## 3	Pentose phosphate pathway	C00022,C00031,C00085,C00117,C00118,C~
## 4	Pentose and glucuronate interconversions	C00022,C00026,C00029,C00103,C00111,C~
## 5	Fructose and mannose metabolism	C00085,C00095,C00096,C00111,C00118,C~
## 6	Galactose metabolism	C00029,C00031,C00052,C00085,C00089,C~

```
cat("\nKEGGset_KEGG:\n")
```

```
##
```

```
## KEGGset_KEGG:
```

```
show(KEGGset_KEGG)
```

```
## EnrichmentSet: KEGG_hsa_pathways
```

```
## Source: KEGG
```

```
## Feature IDs: kegg_id
```

```
## Number of sets: 351
```

```
## Example set: ABC transporters
```

```
print_tbl(head(KEGGset_KEGG@data))
```

```
## # A tibble: 6 x 4
```

##	set_id	set_name	feature_ids	feature_list
----	--------	----------	-------------	--------------


```
##   <chr>   <chr>                                     <chr>         <list>
## 1 PW_1    ABC transporters                          C00009;C00025;C~ <chr [138]>
## 2 PW_10   Aldosterone-regulated sodium reabsorption C00238;C00575;C~ <chr [8]>
## 3 PW_100  Epstein-Barr virus infection              C00076;C01245;C~ <chr [3]>
## 4 PW_101  ErbB signaling pathway                   C00076;C00165;C~ <chr [4]>
## 5 PW_102  Estrogen signaling pathway                C00076;C00165;C~ <chr [8]>
## 6 PW_103  Ether lipid metabolism                   C00670;C00958;C~ <chr [25]>

KEGGset_KEGG_df <- as.MetaboliteSetDataFrame(KEGGset_KEGG, "both")
```

Notice that Metabolite identifiers for this example data are neither HMDB, nor PubChem but KEGG ids!

Building signatures and backgrounds

Our aim is to derive **signatures** and **backgrounds** that are compatible with the `EnrichmentSet` objects, which use **HMDB IDs**.

We therefore:

1. Map each `topTable` to HMDB IDs.
2. Construct a vector of significant HMDB IDs (signature).
3. Use `filterEnrichmentSet(Eset, ids)` to restrict the metabolite sets to those that contain at least one of the signature metabolites.
4. Build a **background** as the union of all metabolites in the filtered sets.

Cachexia: HMDB mapping and signature

```
library(tibble)

## Warning: package 'tibble' was built under R version 4.4.3
# Convert topCachexia to tibble with metabolite name
cachexia_tt <- topCachexia %>%
  rownames_to_column(var = "Metabolite") %>%
  left_join(Cachexia_map, by = c("Metabolite" = "Query"))

cat("Cachexia limma table with mapping:\n")

## Cachexia limma table with mapping:
dim(cachexia_tt)

## [1] 63 16

print_tbl(cachexia_tt)

## # A tibble: 63 x 16
##   Metabolite logFC AveExpr      t P.Value adj.P.Val      B ordByLimma Match HMDB
##   <chr>      <dbl>   <dbl> <dbl>   <dbl>   <dbl> <dbl>   <int> <chr> <chr>
## 1 Quinolate  44.4    66.4  4.06 1.19e-4  0.00426 -3.92      1 Quin~ HMDB~
## 2 Valine     25.4    35.7  3.97 1.64e-4  0.00426 -3.95      2 L-Va~ HMDB~
## 3 N,N-Dimeth~ 20.9    26.3  3.90 2.03e-4  0.00426 -3.97      3 Dime~ HMDB~
## 4 Leucine    17.7    24.4  3.72 3.78e-4  0.00431 -4.02      4 Leuc~ HMDB~
## 5 Dimethylam~ 245.    358.   3.69 4.17e-4  0.00431 -4.03      5 Dime~ HMDB~
## 6 Pyroglutam~ 151.    211.   3.67 4.55e-4  0.00431 -4.04      6 Pyro~ HMDB~
## # i 57 more rows
## # i 6 more variables: PubChem <int>, ChEBI <chr>, KEGG <chr>, METLIN <int>,
```

```
## # SMILES <chr>, Comment <int>
tail(cachexia_tt)

##           Metabolite      logFC AveExpr      t P.Value adj.P.Val
## 58           Acetone   4.926383  11.42701  0.88147387 0.3808486 0.4136804
## 59           Tartrate  18.558681  40.00403  0.76840327 0.4446356 0.4747804
## 60 4-Hydroxyphenylacetate 20.023887 112.02104  0.70996703 0.4799035 0.5038987
## 61           Pantothenate -12.678624  44.88377 -0.62491564 0.5339035 0.5514086
## 62              Uracil   5.020284  35.55766  0.60799978 0.5450055 0.5537959
## 63 1-Methylnicotinamide -2.590745  71.57364 -0.08306038 0.9340225 0.9340225
##           B ordByLimma           Match      HMDB PubChem ChEBI
## 58 -4.609633          58           Acetone HMDB0001659      180 15347
## 59 -4.619378          59      Tartaric acid HMDB0000956  444305 15671
## 60 -4.623907          60 p-Hydroxyphenylacetic acid HMDB0000020      127 18101
## 61 -4.629872          61      Pantothenic acid HMDB0000210     6613 46905
## 62 -4.630970          62              Uracil HMDB0000300     1174 17568
## 63 -4.650155          63 1-Methylnicotinamide HMDB0000699      457 16797
##           KEGG METLIN           SMILES Comment
## 58 C00207   3745           CC(C)=O      1
## 59 C00898    NA   O[C@H]([C@@H](O)C(O)=O)C(O)=O      1
## 60 C00642   130           OC(=O)CC1=CC=C(O)C=C1      1
## 61 C00864    NA  CC(C)(CO)[C@H](O)C(=O)NCCC(O)=O      1
## 62 C00106   258           O=C1NC=CC(=O)N1      1
## 63 C02918   5667      C[N+]1=CC=CC(=C1)C(N)=O      1

# p-value column and threshold
p_col_cachexia <- "adj.P.Val"
alpha_cachexia <- 0.05

# Signature: HMDB IDs of significant metabolites
cachexia_sig <- cachexia_tt %>%
  filter(!is.na(HMDB),
    !!rlang::sym(p_col_cachexia) < alpha_cachexia) %>%
  pull(HMDB) %>%
  unique()

length(cachexia_sig)

## [1] 36

cachexia_sig

## [1] "HMDB0000232" "HMDB0000883" "HMDB0000092" "HMDB0000687" "HMDB0000087"
## [6] "HMDB0000267" "HMDB0000562" "HMDB0000641" "HMDB0000161" "HMDB0000929"
## [11] "HMDB0000211" "HMDB0000043" "HMDB0000167" "HMDB0000164" "HMDB0000187"
## [16] "HMDB0000072" "HMDB0000174" "HMDB0000042" "HMDB0242161" "HMDB0000168"
## [21] "HMDB0000682" "HMDB0000754" "HMDB0000177" "HMDB0000158" "HMDB0000254"
## [26] "HMDB0000094" "HMDB0000875" "HMDB0304356" "HMDB0000243" "HMDB0000479"
## [31] "HMDB0000714" "HMDB0000958" "HMDB0000149" "HMDB0000123" "HMDB0000448"
## [36] "HMDB0000452"
```

Note that in this Cachexia dataset we have mostly (> 50%) *differential* metabolites, although it is hard to say if this is due to the fact that the researches only quantified those metabolites or they only reported those. A realistic background based purely on the data is therefore. Instead, we derive a *functional background* from the metabolite-set collection.

The background will be constructed artificially by taking all sets in the `KEGGset_HMDB` collection and, from there the HMDB ids of each set.

```
# Extract background as the union of all HMDB IDs in the filtered sets
```

```
cachexia_background <- unique(unlist(KEGGset_HMDB@data$feature_list))
length(cachexia_background)
```

```
## [1] 2105
```

```
intersect(cachexia_sig, cachexia_background) |> length()
```

```
## [1] 25
```

We now have all we need to do the Enrichment Analysis on the Cachexia dataset.

- The list of *significant* metabolites: `cachexia_sig`

```
str(cachexia_sig)
```

```
## chr [1:36] "HMDB0000232" "HMDB0000883" "HMDB0000092" "HMDB0000687" ...
```

- The *background* associated with this list: `cachexia_background`

```
str(cachexia_background)
```

```
## chr [1:2105] "HMDB0000243" "HMDB0001206" "HMDB0000042" "HMDB0000223" ...
```

- The collections of metabolite sets to do the enrichment Analysis: `KEGGset_HMDB`.

```
show(KEGGset_HMDB)
```

```
## EnrichmentSet: KEGG_pathways_hsa_HMDB
## Source: KEGG
## Feature IDs: HMDB
## Number of sets: 283
## Example set: Glycolysis / Gluconeogenesis
```

- We can also use `SMPDBset` and `ChemicalClassSet` because these are sets made of HMDB ids.

```
show(SMPDBset)
```

```
## EnrichmentSet: SMPDB_pathways
## Source: SMPDB
## Feature IDs: HMDB
## Number of sets: 48654
## Example set: Citrullinemia Type I
```

```
show(ChemicalClassSet)
```

```
## EnrichmentSet: ChemicalClasses
## Source: HMDB
## Feature IDs: HMDB
## Number of sets: 31
## Example set: Acylcarnitines
```

ST000291: PubChem and HMDB mappings and signatures

ST000291 with PubChem identifiers

The original ST000291 dataset is annotated with PubChem identifiers, so we are restricted to work with Metabolite Sets created with PubChem identifiers.

The code below shows how to prepare a list of selected metabolites and background straight from the available information.

To define the signature we select metabolites with an adjusted p-value of at most 0.33

```
## ST000291: signature (PubChem) and background

# top table from limma
st291_tt <- topCranVSBBase # ja carregat anteriorment

# signature (adj.P.Val < 0.05)
ST291_sig <- st291_tt %>%
  filter(adj_pvalue < 0.33) %>%
  pull(PubChemCID) %>%
  unique()

# background = ALL tested metabolites
ST291_background <- st291_tt %>%
  pull(PubChemCID) %>%
  unique()

cat("Signature size:", length(ST291_sig), "\n")

## Signature size: 50
cat("Background size:", length(ST291_background), "\n")
```

Background size: 1358

ST000291 with HMDB identifiers

Given that many programs can only work with HMDB or KEGG identifiers we have prepared and analyzed a subset of metabolites made only by those that have such ids.

```
## ST000291: signature (HMDB) and background

# top table from limma
st291_tt_HMDB <- topCranVSBBase_HMDB # ja carregat anteriorment

# signature (adj.P.Val < 0.05)
ST291_HMDB_sig <- st291_tt_HMDB %>%
  filter(pvalue < 0.05) %>%
  pull(feature) %>%
  unique()

# background = ALL tested metabolites
ST291_HMDB_background <- st291_tt_HMDB %>%
  pull(feature) %>%
  unique()

cat("Signature size:", length(ST291_HMDB_sig), "\n")

## Signature size: 29
cat("Background size:", length(ST291_HMDB_background), "\n")

## Background size: 623
```

Enrichmet Example: KEGG mappings and signature

The object loaded already contains the signature as EM_inputMetabolites. The background can be obtained from the EM_summaryStats object.

```
EM_sig <- EM_inputMetabolites
EM_background <- EM_summaryStats$met_id
cat("Signature size:", length(EM_sig), "\n")

## Signature size: 57
cat("Background size:", length(EM_background), "\n")

## Background size: 249
```

Enrichment analysis of the Cachexia data

Over Representation Analysis (ORA)

ORA with localEnrichment and KEGGsets_HMDB

We start doing ORA using KEGG metabolite sets with HMDB identifiers

First perform Fisher test for all sets and select the enriched ones.

```
library(localEnrichment)

# ORA per Cachexia vs KEGG (HMDB)

ORA_cachexia_KEGG_LE <- ora_test(
  eset      = KEGGset_HMDB,
  selected  = cachexia_sig,
  background = cachexia_background,
  test      = "fisher",
  p_adjust  = "BH",
  min_set_size = 3,
  max_set_size = 500
)

# veure com és el resultat
dplyr::glimpse(ORA_cachexia_KEGG_LE)

## Rows: 91
## Columns: 8
## $ set_id      <chr> "hsa05230", "hsa00970", "hsa04974", "hsa04978", "hsa~
## $ set_name    <chr> "Central carbon metabolism in cancer", "Aminoacyl-tR~
## $ n_selected_in_set <int> 13, 11, 12, 9, 8, 6, 9, 7, 6, 4, 4, 3, 3, 3, 3, 3~
## $ n_in_set    <int> 35, 23, 47, 27, 47, 26, 82, 45, 40, 15, 17, 10, 12, ~
## $ p_value     <dbl> 1.103121e-15, 8.109981e-15, 2.935896e-12, 1.626280e--
## $ overlap_features <chr> "HMDB0000883;HMDB0000687;HMDB0000641;HMDB0000161;HMD~
## $ p_adj       <dbl> 1.003840e-13, 3.690042e-13, 8.905551e-11, 3.699786e--
## $ fold_enrichment <dbl> 21.718254, 27.964976, 14.929078, 19.490741, 9.952719~

head(ORA_cachexia_KEGG_LE)

## # A tibble: 6 x 8
##   set_id set_name n_selected_in_set n_in_set p_value overlap_features p_adj
##   <chr>   <chr>           <int>    <int>    <dbl> <chr>           <dbl>
```

```
## 1 hsa052~ Central~ 13 35 1.10e-15 HMDB0000883;HMD~ 1.00e-13
## 2 hsa009~ Aminoac~ 11 23 8.11e-15 HMDB0000883;HMD~ 3.69e-13
## 3 hsa049~ Protein~ 12 47 2.94e-12 HMDB0000883;HMD~ 8.91e-11
## 4 hsa049~ Mineral~ 9 27 1.63e-10 HMDB0000883;HMD~ 3.70e- 9
## 5 hsa006~ Glyoxyl~ 8 47 6.32e- 7 HMDB0000641;HMD~ 1.15e- 5
## 6 hsa002~ Alanine~ 6 26 2.92e- 6 HMDB0000641;HMD~ 4.43e- 5
## # i 1 more variable: fold_enrichment <dbl>
```

```
ORA_cachexia_KEGG_LE_sig <- ORA_cachexia_KEGG_LE %>%
  dplyr::filter(p_adj < 0.05) %>%
  dplyr::arrange(p_adj)

nrow(ORA_cachexia_KEGG_LE_sig)
```

```
## [1] 20
```

```
head(ORA_cachexia_KEGG_LE_sig[,1:6], 10)
```

```
## # A tibble: 10 x 6
##   set_id set_name n_selected_in_set n_in_set p_value overlap_features
##   <chr> <chr> <int> <int> <dbl> <chr>
## 1 hsa05230 Central carbon~ 13 35 1.10e-15 HMDB0000883;HMD~
## 2 hsa00970 Aminoacyl-tRNA~ 11 23 8.11e-15 HMDB0000883;HMD~
## 3 hsa04974 Protein digest~ 12 47 2.94e-12 HMDB0000883;HMD~
## 4 hsa04978 Mineral absorp~ 9 27 1.63e-10 HMDB0000883;HMD~
## 5 hsa00630 Glyoxylate and~ 8 47 6.32e- 7 HMDB0000641;HMD~
## 6 hsa00250 Alanine, aspar~ 6 26 2.92e- 6 HMDB0000641;HMD~
## 7 hsa02010 ABC transporte~ 9 82 5.31e- 6 HMDB0000883;HMD~
## 8 hsa00470 D-Amino acid m~ 7 45 6.62e- 6 HMDB0000641;HMD~
## 9 hsa00260 Glycine, serin~ 6 40 4.09e- 5 HMDB0000092;HMD~
## 10 hsa00020 Citrate cycle ~ 4 15 8.61e- 5 HMDB0000072;HMD~
```

This can be visualized by different approaches.

First clean the Pathway names

```
clean_pathway_name <- function(x) {
  return(x)
}

ORA_cachexia_KEGG_LE_sig <- ORA_cachexia_KEGG_LE_sig %>%
  dplyr::mutate(set_name_clean = clean_pathway_name(set_name))

top20 <- ORA_cachexia_KEGG_LE_sig %>% arrange(p_adj)
# %>% dplyr::slice(1:20)

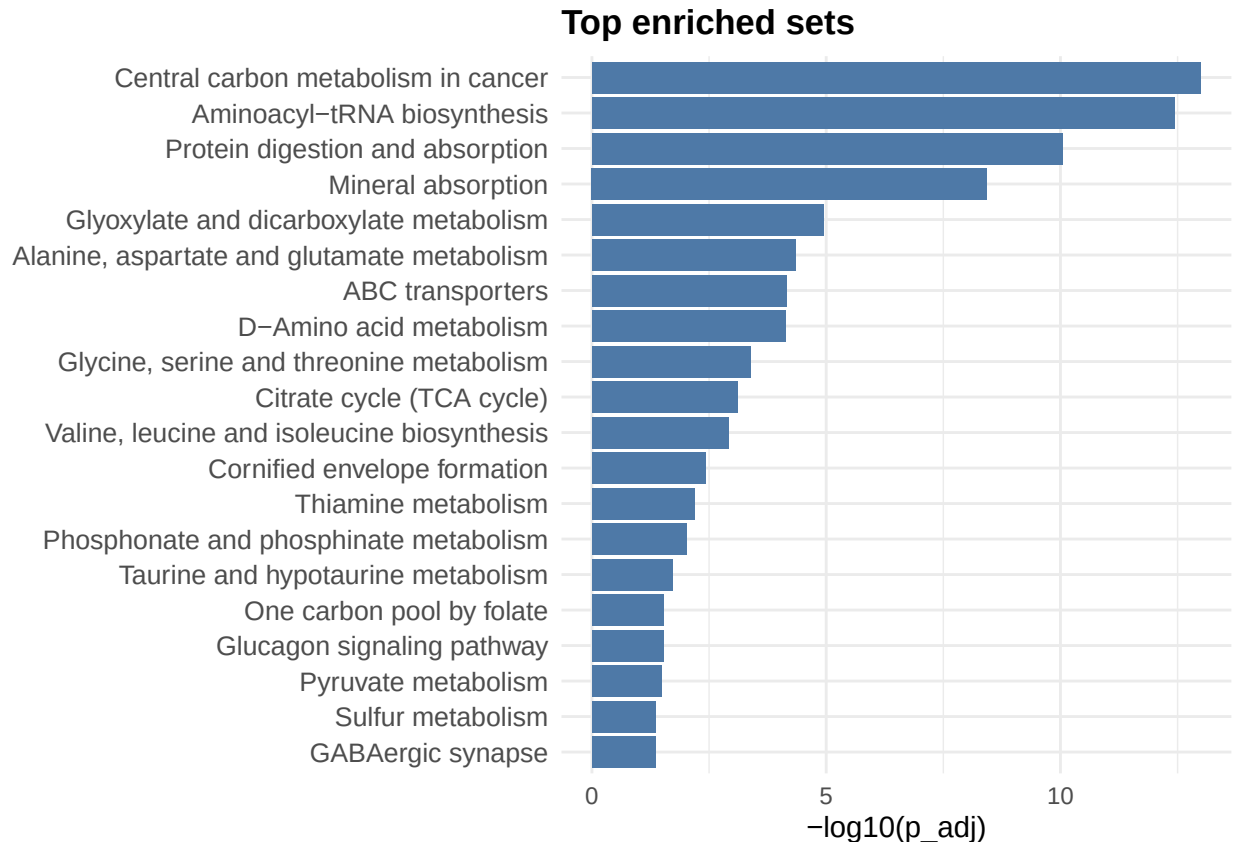
# IMPORTANT: nivells únics per ordenar el gràfic
top20$set_name_clean <- factor(
  top20$set_name_clean,
  levels = rev(unique(top20$set_name_clean))
)

localEnrichment::plot_enrichment(
  results =top20 ,
  type = c("bar", "dot"),
  top = 20,
```

```

x_measure = "-log10(p_adj)",
color_by = "p_adj",
title = NULL
)

```



ORA with `enrichmet` and `KEGGsets_HMDB`

`enrichment` cannot work with `EnrichmentSets` objects -which are defined in `localEnrichment`. Instead it uses a `data.frame` with two columns, “Pathways” and “Metabolites” where metabolites are comma separated IDs.

Such `data.frame` can be extracted from the `EnrichmentSet` as:

```

library(dplyr)

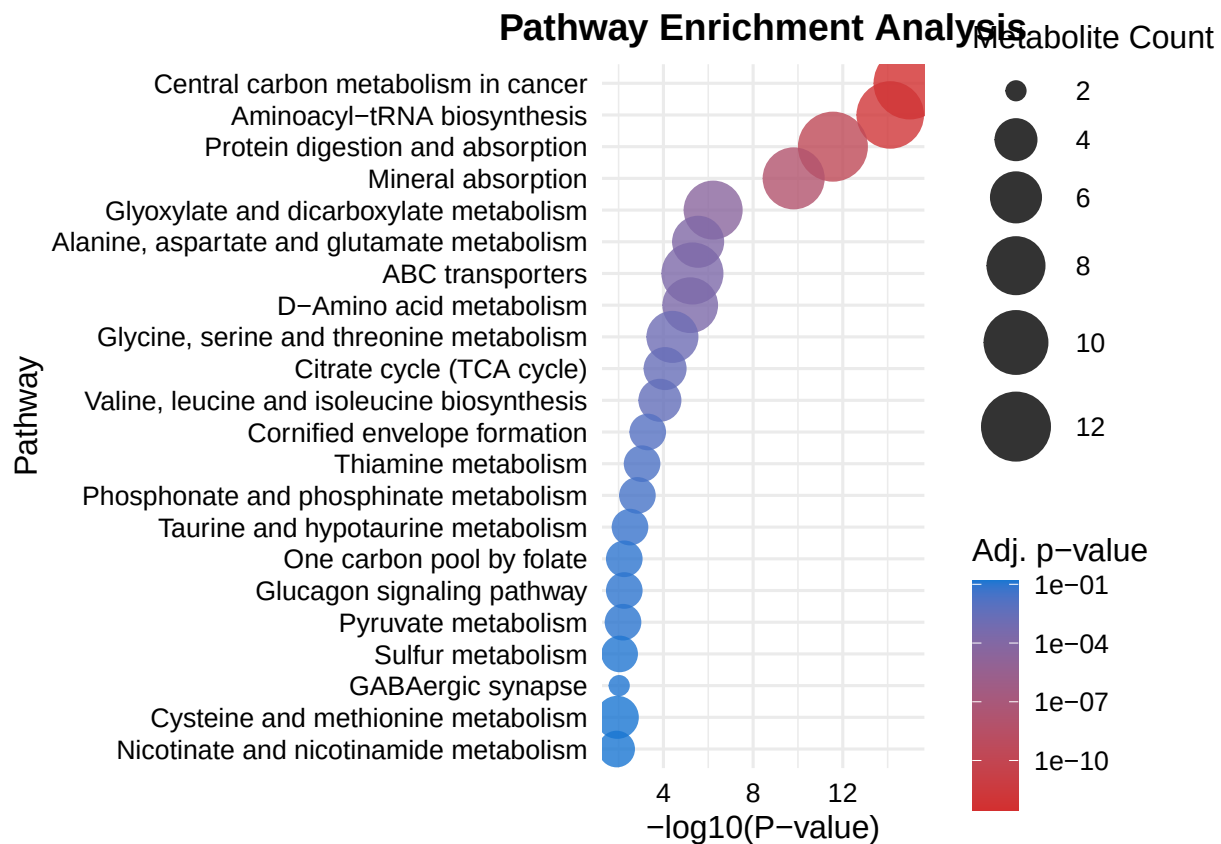
PwVSMet_HMDB <- as.MetaboliteSetDataFrame(
  KEGGset_HMDB,
  id_type = "name",
  id_sep = ",",
) %>%
  dplyr::transmute(
    Pathway = set_name,
    Metabolites = Metabolites
  )

```

Now we can proceed with the `enrichmet` wrapper function:

```
ORA_cachexia_KEGG_EM<- perform_enrichment_analysis(
  inputMetabolites= cachexia_sig,
  PathwayVsMetabolites= PwVSMet_HMDB,
  top_n = 100,
  p_value_cutoff = .25
)

plot <- create_enrichment_plot(ORA_cachexia_KEGG_EM)
plot
```



Metabolite Set Enrichment Analysis

To run MSEA we will use the whole dataset, not only significant metabolites and, additionally, a measure of effect size.

MSEA with localEnrichment and KEGGsets_HMDB

MSEA in localEnrichment has been written to match MetaboAnalyst function so it is also called `gea_test`.

The test requires all the metabolites with an effect size measure.

```
cachexia_scores<- cachexia_tt$t
names(cachexia_scores) <- cachexia_tt$HMDB
```

The test can now be run as:

```
QEA_cachexia_KEGG_LE<- gea_test(
  eset = KEGGset_HMDB,
```



```

scores = cachexia_scores,
nperm = 1000,
p_adjust = "BH",
min_set_size = 3,
max_set_size = 500,
return_profiles = FALSE,
quiet = TRUE
)

```

MSEA with enrichmet and KEGGsets_HMDB

In this case we have to provide a more rigid data structure that can be extracted from cachexia_tt.

```

data4gsea <- cachexia_tt[,c("HMDB", "P.Value", "logFC")]
colnames(data4gsea) <- c("met_id", "pval", "log2fc")

```

```

head(PwVSMet_HMDB, n=10)
head(data4gsea, n=12)

```

```

GSEA_cachexia_KEGG_EM<- perform_gsea_analysis(
  example_data =data4gsea ,
  PathwayVsMetabolites= PwVSMet_HMDB
)
head(GSEA_cachexia_KEGG_EM)

```

MSEA with enrichmet and KEGGsets_KEGG

In this case we must provide a more rigid data structure. Moreover, in some recent versions of **enrichmet** the `perform_gsea_analysis()` function expects KEGG-like identifiers in the `met_id` column, so for this step we use the KEGG annotation instead of HMDB.

First, we build the summary statistics table required by **enrichmet**:

```

data4gsea_KEGG <- cachexia_tt %>%
  dplyr::select(KEGG, P.Value, logFC) %>%
  dplyr::rename(
    met_id = KEGG,
    pval   = P.Value,
    log2fc = logFC
  ) %>%
  dplyr::filter(!is.na(met_id), met_id != "") %>%
  dplyr::distinct(met_id, .keep_all = TRUE)

head(data4gsea_KEGG, 10)

```

```

##    met_id      pval    log2fc
## 1  C03722 0.0001187889  44.42323
## 2  C00183 0.0001640210  25.44989
## 3  C01026 0.0002029160  20.89312
## 4  C00123 0.0003775515  17.70504
## 5  C00543 0.0004169006  244.89730
## 6  C01879 0.0004546884  151.03434
## 7  C00791 0.0004792226 5102.96555
## 8  C00064 0.0009999880  216.98309
## 9  C00041 0.0011130175  190.00706
## 10 C00078 0.0018802610   39.99104

```

We also need a pathway-to-metabolite table using KEGG identifiers:

```
PwVSMet_KEGG <- as.MetaboliteSetDataFrame(
  KEGGset_KEGG,
  id_type = "name",
  id_sep = ",",
) %>%
  dplyr::transmute(
    Pathway = set_name,
    Metabolites = Metabolites
  )

head(PwVSMet_KEGG, 10)
```

```
## # A tibble: 10 x 2
##   Pathway                               Metabolites
##   <chr>                               <chr>
## 1 ABC transporters                   C00009,C00025,C00031,C00032,C00034~
## 2 Aldosterone-regulated sodium reabsorption C00238,C00575,C00698,C00735,C00762~
## 3 Epstein-Barr virus infection       C00076,C01245,C05981
## 4 ErbB signaling pathway             C00076,C00165,C01245,C05981
## 5 Estrogen signaling pathway         C00076,C00165,C00238,C00334,C00533~
## 6 Ether lipid metabolism             C00670,C00958,C01233,C01264,C02773~
## 7 Fanconi anemia pathway            NoMetabolitesfound
## 8 Fat digestion and absorption       C00010,C00040,C00162,C00165,C00187~
## 9 Fatty acid biosynthesis            C00024,C00083,C00154,C00173,C00229~
## 10 Fatty acid degradation            C00010,C00024,C00071,C00136,C00154~
```

Now we can run the enrichmet GSEA-like function:

```
GSEA_cachexia_KEGG_EM <- perform_gsea_analysis(
  example_data = data4gsea_KEGG,
  PathwayVsMetabolites = PwVSMet_KEGG
)
```

```
## |
```

```
head(GSEA_cachexia_KEGG_EM)
```

```
##           pathway           pval      padj      log2err
##           <char>           <num>      <num>      <num>
## 1: Protein_digestion_and_absorption 0.003617475 0.02170485 0.43170770
## 2: Aminoacyl-tRNA_biosynthesis 0.008144665 0.02443400 0.38073040
## 3: Mineral_absorption 0.030075188 0.06015038 0.27128855
## 4: ABC_transporters 0.093814433 0.14072165 0.14463053
## 5: Central_carbon_metabolism_in_cancer 0.173056995 0.20766839 0.10208011
## 6: Glyoxylate_and_dicarboxylate_metabolism 0.485499463 0.48549946 0.05029481
##      ES      NES  size leadingEdge input_count
##      <num>      <num> <int>      <list>      <int>
## 1: 0.6575265 1.7459838   14 C00183, ....      11
## 2: 0.6341130 1.6626901   13 C00183, ....      10
## 3: 0.6221943 1.5812103   11 C00183, ....       8
## 4: 0.5135357 1.3967146   17 C00183, ....       8
## 5: 0.4651647 1.2729082   18 C00183, ....      12
## 6: 0.3908266 0.9932252   11 C00064, ....       7
```

Comparison of results

In any case the results of both tests are relatively similar even if not significant.

```
head(QEA_cachexia_KEGG_LE$table[1:10,c(1,2,6,7)],10)
```

##	set_id	set_name	p_value	p_adj
## 1	hsa04974	Protein digestion and absorption	0.002	0.05200
## 2	hsa04978	Mineral absorption	0.003	0.05200
## 3	hsa00970	Aminoacyl-tRNA biosynthesis	0.004	0.05200
## 4	hsa04976	Bile secretion	0.033	0.32175
## 5	hsa04382	Cornified envelope formation	0.055	0.35850
## 6	hsa00280	Valine, leucine and isoleucine degradation	0.066	0.35850
## 7	hsa05230	Central carbon metabolism in cancer	0.068	0.35850
## 8	hsa02010	ABC transporters	0.088	0.35850
## 9	hsa04922	Glucagon signaling pathway	0.093	0.35850
## 10	hsa00770	Pantothenate and CoA biosynthesis	0.119	0.35850

```
head(GSEA_cachexia_KEGG_EM[,1:3],10)
```

##	pathway	pval	padj
##	<char>	<num>	<num>
## 1:	Protein_digestion_and_absorption	0.003617475	0.02170485
## 2:	Aminoacyl-tRNA_biosynthesis	0.008144665	0.02443400
## 3:	Mineral_absorption	0.030075188	0.06015038
## 4:	ABC_transporters	0.093814433	0.14072165
## 5:	Central_carbon_metabolism_in_cancer	0.173056995	0.20766839
## 6:	Glyoxylate_and_dicarboxylate_metabolism	0.485499463	0.48549946

Enrichment analysis of the enrichmet example data (KRASG12D)

The `enrichmet` package includes a small example dataset derived from a metabolomics comparison involving KRAS G12D mutation. Unlike the Cachexia example, these data are already distributed in a format very close to what enrichment tools require:

- a list of significant metabolites (`EM_inputMetabolites`)
- a summary statistics table (`EM_summaryStats`)
- a pathway-to-metabolite mapping table (`EM_PathwayVsMetabolites`)
- an `EnrichmentSet` object built from the same KEGG-based pathway mapping (`KEGGset_KEGG`)

ORA with localEnrichment and KEGGset_KEGG

We begin by defining the signature and the corresponding background.

```
EM_sig <- unique(EM_inputMetabolites)
EM_background <- unique(EM_summaryStats$met_id)
```

```
cat("Signature size:", length(EM_sig), "\n")
```

```
## Signature size: 57
```

```
cat("Background size:", length(EM_background), "\n")
```

```
## Background size: 249
```

```
intersect(EM_sig, EM_background) |> length()
```

```
## [1] 51
```

The ORA can now be carried out with `localEnrichment::ora_test()` using the KEGG-based `EnrichmentSet`.

```
library(localEnrichment)
library(dplyr)

ORA_EM_KEGG_LE <- ora_test(
  eset = KEGGset_KEGG,
  selected = EM_sig,
  background = EM_background,
  test = "fisher",
  p_adjust = "BH",
  min_set_size = 3,
  max_set_size = 500
)

dplyr::glimpse(ORA_EM_KEGG_LE)

## Rows: 47
## Columns: 8
## $ set_id          <chr> "PW_45", "PW_106", "PW_107", "PW_27", "PW_269", "PW_~
## $ set_name        <chr> "Biosynthesis of unsaturated fatty acids", "Fatty ac~
## $ n_selected_in_set <int> 7, 4, 2, 1, 1, 2, 2, 4, 1, 2, 2, 2, 5, 1, 1, 1, 1, 1~
## $ n_in_set         <int> 11, 7, 3, 14, 15, 4, 4, 11, 13, 5, 5, 5, 33, 10, 10, ~
## $ p_value          <dbl> 0.003633786, 0.050183860, 0.132051780, 0.200320067, ~
## $ overlap_features <chr> "C16527;C00249;C00219;C06429;C06428;C16513;C01530", ~
## $ p_adj            <dbl> 0.1707879, 1.0000000, 1.0000000, 1.0000000, 1.000000~
## $ fold_enrichment  <dbl> 2.7799043, 2.4962406, 2.9122807, 0.3120301, 0.291228~

head(ORA_EM_KEGG_LE)

## # A tibble: 6 x 8
##   set_id set_name      n_selected_in_set n_in_set p_value overlap_features p_adj
##   <chr>  <chr>          <int>      <int>   <dbl>   <chr>          <dbl>
## 1 PW_45  Biosynthesis~           7         11 0.00363 C16527;C00249;C~ 0.171
## 2 PW_106 Fatty acid b~           4          7 0.0502  C00249;C06424;C~ 1
## 3 PW_107 Fatty acid d~           2          3 0.132   C00249;C02990    1
## 4 PW_27  Arginine and~           1         14 0.200   C00148            1
## 5 PW_269 Pyrimidine m~           1         15 0.202   C00295            1
## 6 PW_297 Sphingolipid~          2          4 0.226   C00065;C00836    1
## # i 1 more variable: fold_enrichment <dbl>
```

We may restrict attention to the pathways that remain significant after multiple testing correction.

```
ORA_EM_KEGG_LE_sig <- ORA_EM_KEGG_LE %>%
  dplyr::filter(p_adj < 0.25) %>%
  dplyr::arrange(p_adj)

nrow(ORA_EM_KEGG_LE_sig)

## [1] 1

head(ORA_EM_KEGG_LE_sig[, 1:6], 10)

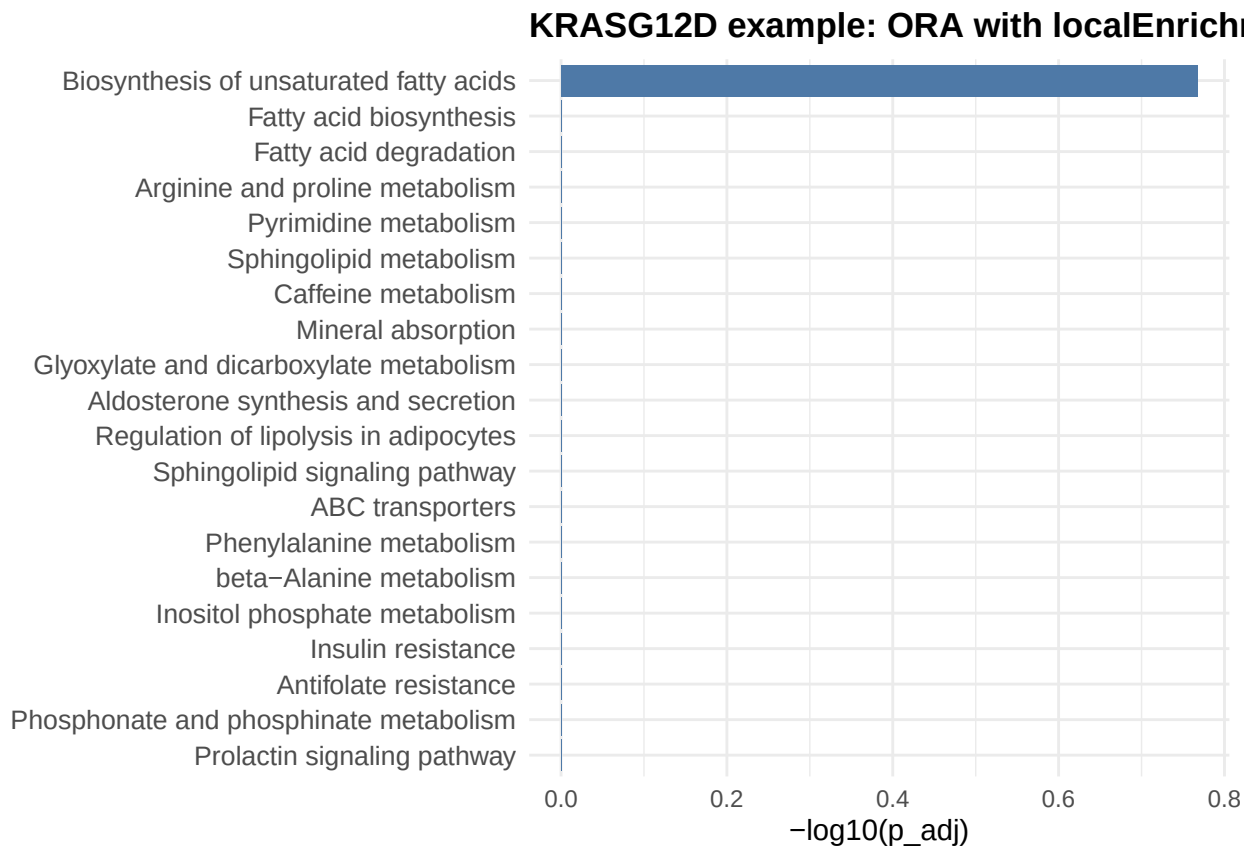
## # A tibble: 1 x 6
##   set_id set_name      n_selected_in_set n_in_set p_value overlap_features
##   <chr>  <chr>          <int>      <int>   <dbl>   <chr>
## 1 PW_45  Biosynthesis of un~           7         11 0.00363 C16527;C00249;C~
```

The enrichment results can be visualized with the standard plotting functions from localEnrichment.

```
top_kras_ora <- ORA_EM_KEGG_LE %>%
  dplyr::arrange(p_adj) %>%
  dplyr::slice(1:20)

top_kras_ora$set_name <- factor(
  top_kras_ora$set_name,
  levels = rev(unique(top_kras_ora$set_name))
)

localEnrichment::plot_enrichment(
  results = top_kras_ora,
  type = c("bar", "dot"),
  top = 20,
  x_measure = "-log10(p_adj)",
  color_by = "p_adj",
  title = "KRASG12D example: ORA with localEnrichment"
)
```



ORA with enrichmet and EM_PathwayVsMetabolites

The same ORA-like analysis can be repeated with enrichmet, using directly the pathway mapping table provided with the example data.

```
ORA_EM_KEGG_EM <- perform_enrichment_analysis(
  inputMetabolites = EM_sig,
```

```

PathwayVsMetabolites = EM_PathwayVsMetabolites,
top_n = 100,
p_value_cutoff = 0.25
)

head(ORA_EM_KEGG_EM)

```

```

##              Pathway      P_value Log_P_value      Impact
## 1  Central carbon metabolism in cancer 4.817117e-05  4.317213 0.05461853
## 2    Protein digestion and absorption 1.925909e-04  3.715364 0.06838474
## 3 Biosynthesis of unsaturated fatty acids 3.699903e-04  3.431810 0.70197902
## 4          Caffeine metabolism 6.220368e-04  3.206184 0.09487611
## 5          Mineral absorption 1.832389e-03  2.736982 0.09229047
##      Coverage Count Adjusted_P_value      Q_value
## 1 0.16216216      6      0.01690808 0.0003367180
## 2 0.12765957      6      0.03379971 0.0006731084
## 3 0.09459459      7      0.04328886 0.0008620812
## 4 0.18181818      4      0.05458373 0.0010870143
## 5 0.13793103      4      0.12863372 0.0025616917

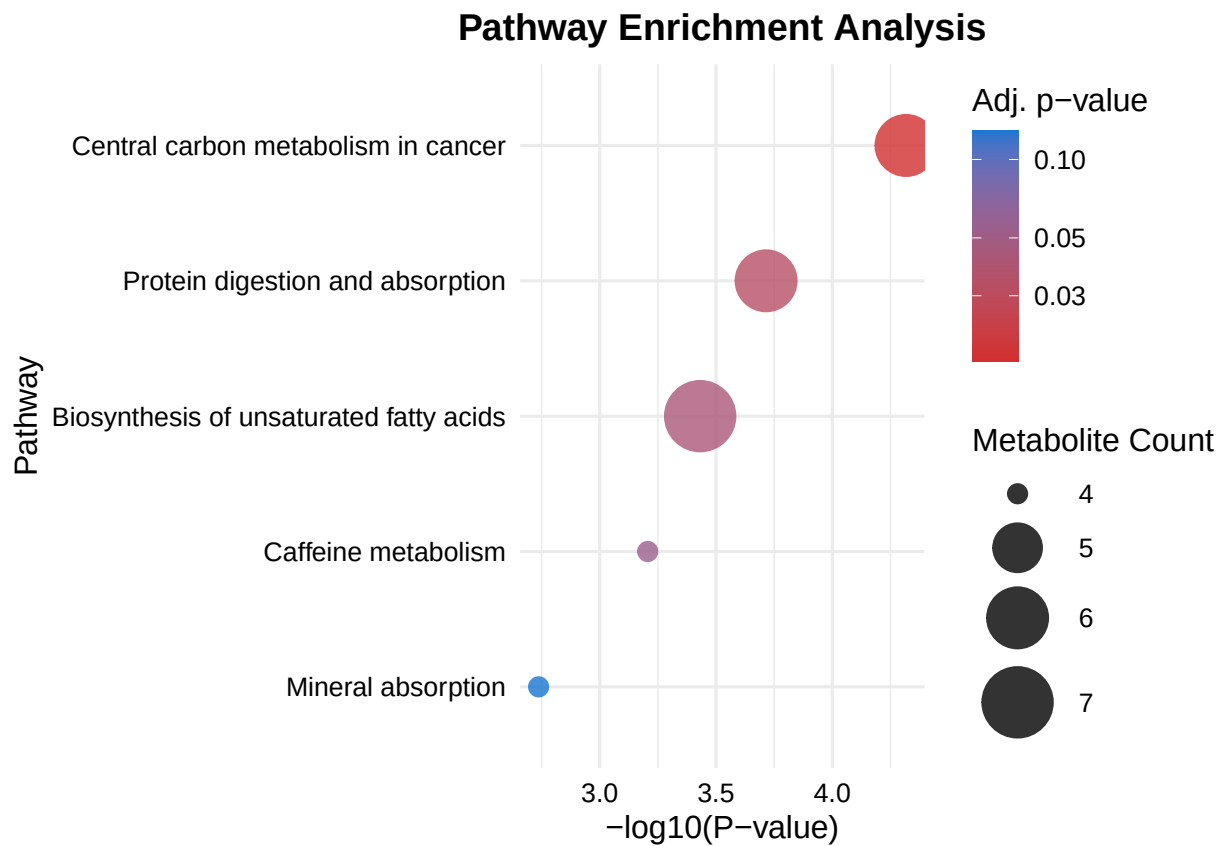
```

And visualized with the plotting function supplied by enrichmet.

```

plot <- create_enrichment_plot(ORA_EM_KEGG_EM)
plot

```



Because both analyses use the same KEGG identifiers and a pathway mapping derived from the same source, this example is particularly convenient to compare the behaviour of the two implementations.

MSEA / QEA with localEnrichment and KEGGset_KEGG

To run a quantitative enrichment analysis with localEnrichment, we need a numeric score for all metabolites in the ranked list. Here we will use the log2fc column from EM_summaryStats.

```
EM_scores <- EM_summaryStats$log2fc
names(EM_scores) <- EM_summaryStats$met_id

summary(EM_scores)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -4.7235 -0.7930 -0.1904 -0.2540  0.4430  3.1852
```

```
head(EM_scores)
```

```
##      C02721      C03736      C03139      C00074      C02862      C00386
## -2.324807  2.634161  1.748349 -3.219306 -1.697477  1.683674
```

The GSEA-like quantitative enrichment analysis can then be run with qea_test().

```
QEA_EM_KEGG_LE <- qea_test(
  eset = KEGGset_KEGG,
  scores = EM_scores,
  nperm = 1000,
  p_adjust = "BH",
  min_set_size = 3,
  max_set_size = 500,
  return_profiles = FALSE,
  quiet = TRUE
)

head(QEA_EM_KEGG_LE$table[, c("set_id", "set_name", "p_value", "p_adj")], 10)
```

```
##      set_id                set_name p_value    p_adj
## 1  PW_263      Protein digestion and absorption  0.000 0.00000000
## 2  PW_269      Pyrimidine metabolism  0.001 0.04250000
## 3  PW_111      Ferroptosis  0.002 0.05666667
## 4  PW_16      Aminoacyl-tRNA biosynthesis  0.004 0.08500000
## 5  PW_195      Mineral absorption  0.008 0.13600000
## 6  PW_85      D-Amino acid metabolism  0.016 0.22666667
## 7  PW_45      Biosynthesis of unsaturated fatty acids  0.023 0.27928571
## 8  PW_86      Diabetic cardiomyopathy  0.031 0.30222222
## 9  PW_81      Cysteine and methionine metabolism  0.032 0.30222222
## 10 PW_326      Tryptophan metabolism  0.037 0.31450000
```

If desired, we can inspect only the most significant pathways.

```
QEA_EM_KEGG_LE_sig <- QEA_EM_KEGG_LE$table %>%
  dplyr::filter(p_adj < 0.25) %>%
  dplyr::arrange(p_adj)

nrow(QEA_EM_KEGG_LE_sig)
```

```
## [1] 6
```

```
head(QEA_EM_KEGG_LE_sig[, c("set_id", "set_name", "p_value", "p_adj")], 10)
```

```
##      set_id                set_name p_value    p_adj
## 1  PW_263      Protein digestion and absorption  0.000 0.00000000
```

```
## 2 PW_269          Pyrimidine metabolism  0.001 0.04250000
## 3 PW_111          Ferroptosis           0.002 0.05666667
## 4 PW_16           Aminoacyl-tRNA biosynthesis 0.004 0.08500000
## 5 PW_195          Mineral absorption     0.008 0.13600000
## 6 PW_85           D-Amino acid metabolism 0.016 0.22666667
```

MSEA / GSEA with enrichmet

The enrichmet package already provides a function to perform a GSEA-like analysis from the summary statistics table.

```
GSEA_EM_KEGG_EM <- perform_gsea_analysis(
  example_data = EM_summaryStats,
  PathwayVsMetabolites = EM_PathwayVsMetabolites
)
```

```
## |
head(GSEA_EM_KEGG_EM[, 1:3], 10)
```

```
##           pathway           pval      padj
##           <char>         <num>    <num>
##  1:           Mineral_absorption 0.001028898 0.01105458
##  2: Protein_digestion_and_absorption 0.001381823 0.01105458
##  3:           Aminoacyl-tRNA_biosynthesis 0.005388715 0.02873981
##  4: Central_carbon_metabolism_in_cancer 0.016363398 0.06545359
##  5:           D-Amino_acid_metabolism 0.029433062 0.09418580
##  6:           ABC_transporters 0.096153846 0.24024024
##  7:           Pyrimidine_metabolism 0.111111111 0.24024024
##  8: Biosynthesis_of_unsaturated_fatty_acids 0.133211679 0.24024024
##  9: Cysteine_and_methionine_metabolism 0.135135135 0.24024024
## 10: Arginine_and_proline_metabolism 0.167259786 0.26761566
```

This allows a direct comparison with the localEnrichment implementation, since both analyses are now based on the same ranked metabolite list and the same KEGG pathway definitions.

Quick comparison of the top pathways

A simple way to compare both implementations is to inspect the top ranked pathways side by side.

```
top_qea_le <- QEA_EM_KEGG_LE$table %>%
  dplyr::arrange(p_adj) %>%
  dplyr::select(set_name, p_value, p_adj) %>%
  dplyr::slice(1:10) %>%
  dplyr::rename(
    pathway = set_name,
    p_value_LE = p_value,
    p_adj_LE = p_adj
  )

top_gsea_em <- GSEA_EM_KEGG_EM %>%
  as_tibble() %>%
  dplyr::arrange(padj) %>%
  dplyr::select(pathway, pval, padj) %>%
  dplyr::slice(1:10) %>%
  dplyr::rename(
    p_value_EM = pval,
```



```

    p_adj_EM = padj
  )

top_qea_le

##                pathway p_value_LE  p_adj_LE
## 1  Protein digestion and absorption    0.000 0.00000000
## 2      Pyrimidine metabolism          0.001 0.04250000
## 3      Ferroptosis                   0.002 0.05666667
## 4  Aminoacyl-tRNA biosynthesis        0.004 0.08500000
## 5      Mineral absorption            0.008 0.13600000
## 6  D-Amino acid metabolism           0.016 0.22666667
## 7  Biosynthesis of unsaturated fatty acids 0.023 0.27928571
## 8      Diabetic cardiomyopathy        0.031 0.30222222
## 9  Cysteine and methionine metabolism  0.032 0.30222222
## 10     Tryptophan metabolism          0.037 0.31450000

top_gsea_em

```

```

## # A tibble: 10 x 3
##   pathway                p_value_EM p_adj_EM
##   <chr>                  <dbl>    <dbl>
## 1 Mineral_absorption      0.00103  0.0111
## 2 Protein_digestion_and_absorption 0.00138  0.0111
## 3 Aminoacyl-tRNA_biosynthesis 0.00539  0.0287
## 4 Central_carbon_metabolism_in_cancer 0.0164  0.0655
## 5 D-Amino_acid_metabolism  0.0294  0.0942
## 6 ABC_transporters         0.0962  0.240
## 7 Pyrimidine_metabolism    0.111    0.240
## 8 Biosynthesis_of_unsaturated_fatty_acids 0.133  0.240
## 9 Cysteine_and_methionine_metabolism 0.135  0.240
## 10 Arginine_and_proline_metabolism 0.167  0.268

```

In this KRASG12D example the compatibility between both tools is cleaner than in the Cachexia case, because:

the identifiers are already KEGG compound identifiers,

the pathway mapping used by enrichmet is directly available,

the EnrichmentSet object used by localEnrichment has been built from the same source.

This makes the example particularly useful as a controlled use case to compare the behaviour of the two packages under nearly identical inputs.

Appendix I Methods and packages explained

Over-Representation Analysis (ORA)

Concept

Over-Representation Analysis (ORA) is a classical approach used to identify biological sets (such as pathways, chemical classes, or ontological categories) that are statistically *enriched* in a predefined list of significant features, typically metabolites or genes.

The key idea is to test whether a particular set contains **more significant features than expected by chance**, given a specified background.

For each set, a contingency table is constructed:

	In Set	Not in Set	Total
Selected (hits)	k	$K - k$	K
Not selected	$m - k$	$N - m - K + k$	$N - K$

where: - $\mathbf{N}$ = total number of features in the background - $\mathbf{K}$ = number of selected (significant) features
- $\mathbf{m}$ = number of features in the given set
- $\mathbf{k}$ = number of selected features that belong to that set

A statistical test (typically **Fisher’s exact test** or the **hypergeometric test**) is then applied to compute the probability of observing at least k selected features in the set under a random sampling model.

The resulting p -values are adjusted for multiple testing (e.g., Benjamini–Hochberg FDR correction), and the **fold enrichment** is often reported as:

$$\text{Fold Enrichment} = \frac{k/K}{m/N}$$

indicating how much more represented the set is among selected features than expected by chance.

Implementation in `localEnrichment::ora_test()`

The function `ora_test()` implements this classical ORA workflow in a flexible and reproducible way using the `EnrichmentSet` object as input. It can be applied to any type of feature-to-set mapping, including pathways, metabolite classes, or ontological categories.

Quantitative Enrichment Analysis (QEA / MSEA)

Concept

Quantitative Enrichment Analysis (QEA), also known as **Metabolite Set Enrichment Analysis (MSEA)**, is the analogue of GSEA for metabolomics and other continuous-valued omics data.

Instead of dichotomizing features into “significant/non-significant,” QEA uses **the entire ranked list** of features, each with an associated score such as:

- log fold change
- t-statistic
- correlation coefficient
- loading from PCA/MFA/PLS
- any quantity that defines an ordering of features

For each metabolite set, QEA tests whether its members tend to appear **towards the top or bottom** of the ranked list more often than expected by chance.

This avoids arbitrary cutoffs and provides sensitivity to subtle but consistent shifts across a pathway or chemical class.

MSEA variant implemented in localEnrichment

There are several variants of MSEA/QEA in the literature:

1. **ORA-like MSEA** (MetaboAnalyst’s early implementation): uses Fisher tests on selected metabolites → *not what we use here*.
2. **SSPA / Mean/median tests** (subset-based statistics): compares mean scores inside vs. outside a set → *not used here*.
3. **Correlation-based global tests** (e.g., GlobalTest): model score–membership associations → *not used here*.
4. **GSEA-style running-sum statistic** (Subramanian et al., PNAS 2005): uses an enrichment score based on a weighted random walk and permutation testing: **this is the method implemented in qea_test()**.

localEnrichment implements a *GSEA-style running-score MSEA*, adapted to metabolomics.

Specifically:

- The ranked list is used **without thresholding**.
- For each set, a **running enrichment profile** is computed:
 - hits contribute $+1/N_h$
 - misses contribute $-1/(N - N_h)$
- The Enrichment Score (**ES**) is the maximum (positive enrichment) or minimum (negative enrichment) deviation from zero.
- Significance is evaluated by **permuting feature positions**, exactly as in classical GSEA.

This is the most statistically robust form of QEA/MSEA and is the same approach used in modern tools such as **fgsea**, **clusterProfiler::GSEA()**, and the updated **MetaboAnalyst 5.0 “QEA” module**.

Resources and references

Resources

- The tidymass project

-Data Analysis in Metabolomics

References